

TypeScript Fundamentals - Part 2

Data Types in TypeScript

Understanding npm vs npx

Before we dive into TypeScript data types, let's address an important setup consideration. In some corporate environments, global package installation with `npm install -g` may be restricted due to security policies. Understanding the difference between npm and npx helps you work around this limitation.

What is npm?

npm (Node Package Manager) is a package manager for JavaScript. It serves two main purposes: installing packages and managing dependencies in your projects.

When you use `npm install`, packages are installed either globally (available system-wide) or locally (only in your project):

```
// Global installation - requires admin/elevated permissions
npm install -g typescript

// Local installation - installs in current project only
npm install typescript
```

Output:

```
added 1 package in 1.5s
```

Global installations make commands available everywhere. Local installations add packages to `node_modules` in your project.

What is npx?

npx (Node Package Execute) is a package runner that comes bundled with npm (version 5.2.0 and above). It allows you to execute packages without installing them globally.

npx works in two ways: it can run locally installed packages, or it can temporarily download and execute packages without permanent installation.

```
// Using npx to run TypeScript compiler from local installation
npx tsc --version

// npx can also run packages not installed at all
npx create-react-app my-app
```

Output:

```
Version 5.3.3
```

npx finds the package in your local node_modules or downloads it temporarily.

Key Differences Between npm and npx

Feature	npm	npx
Primary Purpose	Install and manage packages	Execute packages
Global Installation	Requires -g flag and permissions	Not needed
Disk Space	Packages stored permanently	Can run without storing
Version Control	Uses installed version	Can specify version on-the-fly
Use Case	Adding dependencies to projects	Running commands or one-time tools

Working with TypeScript When Global Installation is Blocked

If your organization blocks global installations, you can still use TypeScript effectively. Here's the recommended approach:

Step 1: Install TypeScript locally in your project

```
npm install typescript --save-dev
```

Output:

```
added 1 package, and audited 2 packages in 2s  
found 0 vulnerabilities
```

The --save-dev flag adds TypeScript to your project's devDependencies.

Step 2: Use npx to run the TypeScript compiler

```
npx tsc --version  
npx tsc myfile.ts  
npx tsc --init
```

Output:

```
Version 5.3.3  
Created a new tsconfig.json
```

npx executes the locally installed TypeScript compiler without needing global installation.

Step 3: Add scripts to package.json for convenience

```
{  
  "name": "my-project",  
  "scripts": {  
    "build": "tsc",  
  }  
}
```

```
"watch": "tsc --watch",  
"compile": "tsc"  
},  
"devDependencies": {  
  "typescript": "^5.3.3"  
}  
}
```

Now you can run TypeScript commands using npm scripts:

```
npm run build  
npm run watch
```

Output:

```
Starting compilation in watch mode...
```

npm scripts automatically use locally installed packages, no npx needed.

Note:

Best Practice: Even when global installation is allowed, installing TypeScript locally per project is recommended. This ensures that everyone working on the project uses the same TypeScript version, avoiding compatibility issues.

TypeScript Data Types

TypeScript's type system is the foundation of its power. Every value in TypeScript has a type, which tells the compiler what operations are valid on that value. Understanding these types is essential for writing reliable TypeScript code.

Primitive Data Types

Primitive types are the basic building blocks of TypeScript. They represent single values and are immutable.

String Type

The `string` type represents textual data. Strings can be enclosed in single quotes, double quotes, or backticks (for template literals).

```
let firstName: string = "John";
let lastName: string = 'Doe';
let fullName: string = `${firstName} ${lastName}`;
let greeting: string = `Hello, ${fullName}!`;

console.log(firstName);
console.log(lastName);
console.log(fullName);
console.log(greeting);
```

Output:

```
John
Doe
John Doe
Hello, John Doe!
```

In this example, we declare variables with explicit `string` type annotations. The backtick syntax allows us to embed expressions inside strings using `${}`, which is called template literal interpolation.

Number Type

The `number` type represents both integer and floating-point numbers. TypeScript does not distinguish between different numeric types like some languages do.

```
let age: number = 25;
let price: number = 99.99;
let temperature: number = -5;
```

```
let hexValue: number = 0xf00d;
let binaryValue: number = 0b1010;
let octalValue: number = 0o744;

console.log("Age:", age);
console.log("Price:", price);
console.log("Temperature:", temperature);
console.log("Hex:", hexValue);
console.log("Binary:", binaryValue);
console.log("Octal:", octalValue);
```

Output:

```
Age: 25
Price: 99.99
Temperature: -5
Hex: 61453
Binary: 10
Octal: 484
```

TypeScript supports decimal, hexadecimal (0x), binary (0b), and octal (0o) number literals.

Boolean Type

The `boolean` type represents logical values: `true` or `false`. Booleans are commonly used for conditional logic and control flow.

```
let isActive: boolean = true;
let hasPermission: boolean = false;
let isGreater: boolean = 10 > 5;

console.log("Is Active:", isActive);
console.log("Has Permission:", hasPermission);
console.log("Is 10 > 5:", isGreater);

if (isActive) {
    console.log("User is active");
}
```

```
} else {  
    console.log("User is inactive");  
}
```

Output:

```
Is Active: true  
Has Permission: false  
Is 10 > 5: true  
User is active
```

Complex Data Types

Array Type

Arrays store collections of elements of the same type. TypeScript provides two syntaxes for declaring array types.

```
// Syntax 1: Type[]  
let numbers: number[] = [1, 2, 3, 4, 5];  
let names: string[] = ["Alice", "Bob", "Charlie"];  
  
// Syntax 2: Array<Type>  
let scores: Array<number> = [85, 90, 78, 92];  
let cities: Array<string> = ["Mumbai", "Delhi", "Bangalore"];  
  
console.log("Numbers:", numbers);  
console.log("Names:", names);  
console.log("Scores:", scores);  
console.log("Cities:", cities);  
  
// Array methods  
console.log("First number:", numbers[0]);  
console.log("Array length:", names.length);  
numbers.push(6);  
console.log("After push:", numbers);
```

Output:

```
Numbers: [ 1, 2, 3, 4, 5 ]
Names: [ 'Alice', 'Bob', 'Charlie' ]
Scores: [ 85, 90, 78, 92 ]
Cities: [ 'Mumbai', 'Delhi', 'Bangalore' ]
First number: 1
Array length: 3
After push: [ 1, 2, 3, 4, 5, 6 ]
```

Both syntaxes are equivalent. The `Type[]` syntax is more commonly used. TypeScript ensures that you can only add elements of the correct type to the array.

```
let items: number[] = [1, 2, 3];
// items.push("hello"); // ✗ Error
items.push(4); // ✓ Correct
```

Object Type

Objects store collections of key-value pairs. You can specify the structure of an object using type annotations.

```
let user: { name: string; age: number; email: string } = {
  name: "Rahul",
  age: 28,
  email: "rahul@example.com"
};

let product: { id: number; title: string; price: number } = {
  id: 101,
  title: "Laptop",
  price: 45000
};

console.log("User:", user);
console.log("User Name:", user.name);
console.log("Product:", product);
console.log("Product Price:", product.price);
```


Output:

```
User: { name: 'Rahul', age: 28, email: 'rahul@example.com' }  
User Name: Rahul  
Product: { id: 101, title: 'Laptop', price: 45000 }  
Product Price: 45000
```

Tuple Type

Tuples are arrays with a fixed number of elements where each element can have a different type. The order and type of elements are strictly enforced.

```
// Tuple with string and number  
let person: [string, number] = ["Alice", 25];  
  
// Tuple with multiple types  
let transaction: [number, string, boolean, number] = [  
    12345,  
    "Purchase",  
    true,  
    299.99  
];  
  
console.log("Person:", person);  
console.log("Name:", person[0]);  
console.log("Age:", person[1]);  
  
console.log("Transaction:", transaction);  
console.log("Transaction ID:", transaction[0]);  
console.log("Type:", transaction[1]);  
console.log("Success:", transaction[2]);  
console.log("Amount:", transaction[3]);
```

Output:

```
Person: [ 'Alice', 25 ]  
Name: Alice  
Age: 25  
Transaction: [ 12345, 'Purchase', true, 299.99 ]
```

Transaction ID: 12345

Type: Purchase

Success: true

Amount: 299.99

Tuples are useful when you need to return multiple values from a function or when you want to ensure a specific structure for an array.

```
// Practical example: coordinate system
let coordinates: [number, number] = [23.5, 87.3];
console.log(`Latitude: ${coordinates[0]}, Longitude: ${coordinates[1]}`);
```

Output:

Latitude: 23.5, Longitude: 87.3

Enum Type

Enums allow you to define a set of named constants. They make code more readable by replacing magic numbers or strings with meaningful names.

Numeric Enums:

```
enum Direction {
    Up,
    Down,
    Left,
    Right
}

enum Status {
    Pending = 1,
    Approved = 2,
    Rejected = 3
}

let currentDirection: Direction = Direction.Up;
let orderStatus: Status = Status.Approved;
```

```
console.log("Direction.Up:", Direction.Up);
console.log("Direction.Down:", Direction.Down);
console.log("Current Direction:", currentDirection);
console.log("Order Status:", orderStatus);
```

Output:

```
Direction.Up: 0
Direction.Down: 1
Current Direction: 0
Order Status: 2
```

Numeric enums start at 0 by default, but you can set custom values.

String Enums:

```
enum Color {
    Red = "RED",
    Green = "GREEN",
    Blue = "BLUE"
}

enum LogLevel {
    Error = "ERROR",
    Warning = "WARNING",
    Info = "INFO",
    Debug = "DEBUG"
}

let favoriteColor: Color = Color.Blue;
let currentLogLevel: LogLevel = LogLevel.Info;

console.log("Favorite Color:", favoriteColor);
console.log("Log Level:", currentLogLevel);
```

Output:

Favorite Color: BLUE

Log Level: INFO

String enums provide more meaningful runtime values.

Special Types

Any Type

The `any` type is an escape hatch from TypeScript's type checking. A variable with type `any` can hold any value, and TypeScript will not check its usage.

```
let dynamicValue: any = "Hello";
console.log("Initial value:", dynamicValue);

dynamicValue = 42;
console.log("Changed to number:", dynamicValue);

dynamicValue = true;
console.log("Changed to boolean:", dynamicValue);

dynamicValue = { name: "Test" };
console.log("Changed to object:", dynamicValue);
```

Output:

```
Initial value: Hello
Changed to number: 42
Changed to boolean: true
Changed to object: { name: 'Test' }
```

Note:

Important: Use `any` sparingly. It defeats the purpose of TypeScript's type safety. Only use it when you're migrating JavaScript code or dealing with truly dynamic data where the type is unknown.

Void Type

The `void` type represents the absence of a return value. It is commonly used for functions that perform actions but do not return anything.

```
function logMessage(message: string): void {  
    console.log("Log:", message);  
}  
  
function displayAlert(title: string, body: string): void {  
    console.log(`Alert: ${title}`);  
    console.log(`Message: ${body}`);  
}  
  
logMessage("Application started");  
displayAlert("Welcome", "Thank you for using our app");
```

Output:

```
Log: Application started  
Alert: Welcome  
Message: Thank you for using our app
```

Never Type

The `never` type represents values that never occur. It is used for functions that never return (either because they throw an error or run indefinitely).

```
function throwError(message: string): never {  
    throw new Error(message);  
}  
  
function processValue(value: string | number): string {  
    if (typeof value === "string") {  
        return value.toUpperCase();  
    } else if (typeof value === "number") {  
        return value.toString();  
    } else {  
        const exhaustiveCheck: never = value;  
        return exhaustiveCheck;  
    }  
}
```

```
}  
}  
  
console.log(processValue("hello"));  
console.log(processValue(123));
```

Output:

```
HELLO  
123
```

The never type is useful for exhaustiveness checking in conditional logic.

Null and Undefined

TypeScript has two special types for representing absence of value: `null` and `undefined`.

```
let nullValue: null = null;  
let undefinedValue: undefined = undefined;  
  
console.log("Null value:", nullValue);  
console.log("Undefined value:", undefinedValue);  
  
// Practical example  
let userName: string | null = null;  
console.log("User name before:", userName);  
  
userName = "Priya";  
console.log("User name after:", userName);
```

Output:

```
Null value: null  
Undefined value: undefined  
User name before: null  
User name after: Priya
```

Advanced Type Features

Union Types

Union types allow a variable to hold values of multiple types. You use the pipe symbol (`|`) to combine types.

```
let id: string | number;

id = "ABC123";
console.log("ID as string:", id);

id = 12345;
console.log("ID as number:", id);

function displayId(id: string | number): void {
  if (typeof id === "string") {
    console.log("String ID:", id.toUpperCase());
  } else {
    console.log("Numeric ID:", id.toFixed(2));
  }
}

displayId("xyz789");
displayId(456.789);
```

Output:

```
ID as string: ABC123
ID as number: 12345
String ID: XYZ789
Numeric ID: 456.79
```

Type Aliases

Type aliases allow you to create custom names for types. This makes complex types more readable and reusable.

```
type ID = string | number;

type User = {
  id: ID;
  name: string;
  email: string;
  age: number;
};

type Product = {
  productId: ID;
  title: string;
  price: number;
  inStock: boolean;
};

let user1: User = {
  id: "U001",
  name: "Sneha",
  email: "sneha@example.com",
  age: 27
};

let product1: Product = {
  productId: 501,
  title: "Smartphone",
  price: 25000,
  inStock: true
};

console.log("User:", user1);
console.log("Product:", product1);
```

Output:

```
User: { id: 'U001', name: 'Sneha', email: 'sneha@example.com', age: 27 }
Product: { productId: 501, title: 'Smartphone', price: 25000, inStock: true }
```

Type aliases can also be used for function types:


```
type MathOperation = (a: number, b: number) => number;

const add: MathOperation = (a, b) => a + b;
const multiply: MathOperation = (a, b) => a * b;

console.log("Add:", add(5, 3));
console.log("Multiply:", multiply(5, 3));
```

Output:

```
Add: 8
Multiply: 15
```

Type Assertions

Type assertions allow you to tell the TypeScript compiler that you know more about the type of a value than it does. This is sometimes called "type casting" in other languages.

```
// Using "as" syntax
let someValue: any = "this is a string";
let strLength: number = (someValue as string).length;
console.log("String length:", strLength);

// Another example
let input: unknown = "Hello TypeScript";
let inputLength: number = (input as string).length;
console.log("Input length:", inputLength);

// Practical example with DOM
let myCanvas = document.getElementById("canvas") as HTMLCanvasElement;
// Now TypeScript knows myCanvas is specifically an HTMLCanvasElement
```

Output:

```
String length: 16
Input length: 16
```

Type assertions don't change the runtime value, they only tell TypeScript how to treat the type.

Note:

Important: Type assertions should be used carefully. They bypass TypeScript's type checking, so you're responsible for ensuring the assertion is correct. Incorrect assertions can lead to runtime errors.

Summary

In this section, we covered TypeScript's comprehensive type system. You learned about primitive types (string, number, boolean), complex types (arrays, objects, tuples, enums), special types (any, void, never, null, undefined), and advanced features (union types, type aliases, type assertions).

We also addressed the practical difference between npm and npx, which is crucial when working in environments with restricted global installations. Remember that TypeScript's type system is designed to catch errors early and make your code more maintainable.

Key takeaways: use specific types whenever possible, avoid `any` unless necessary, leverage union types for flexibility, and use type aliases to make complex types more readable. In the next section, we'll explore how to use these types with functions and learn about type annotations and inference in detail.